# MJQbe Web

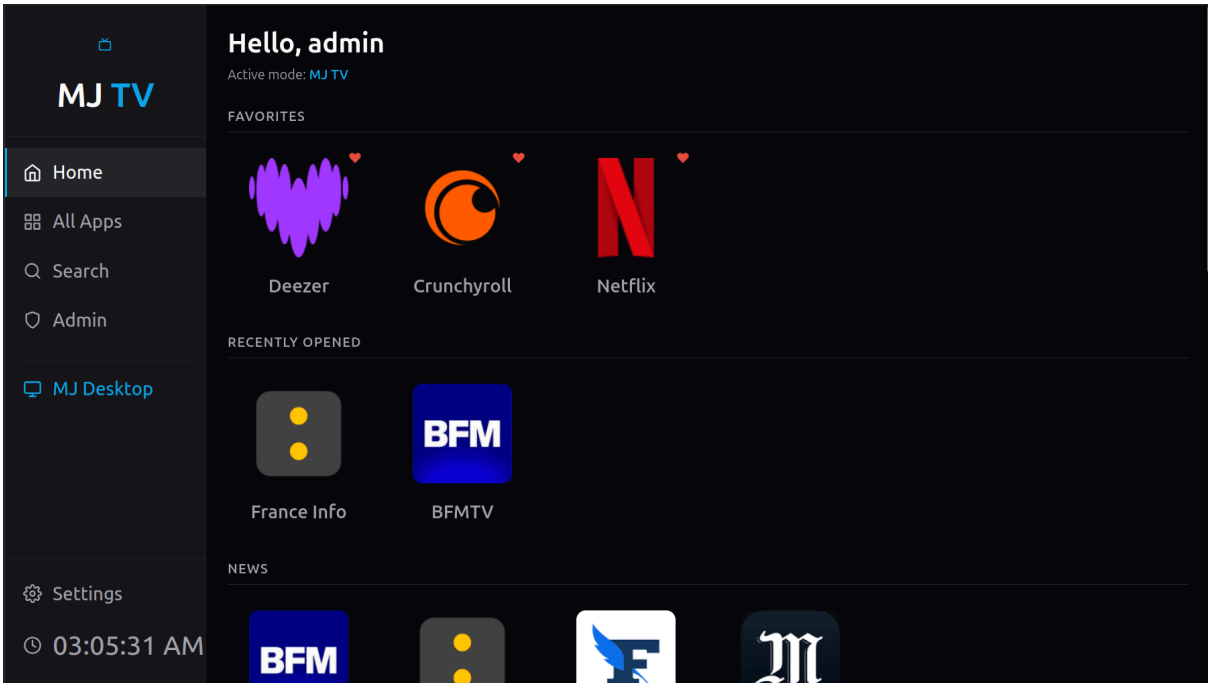
## TP Développement Fullstack

BARBIER Jade

4A Info Alt

# Sommaire

**Sommaire** ..... **2**
**1. Description du système** ..... **3**
1.1. Contexte et objectif ..... 3
1.2. Fonctionnalités ..... 3
1.3. Expérience utilisateur ..... 4
1.4. Stratégies FullStack ..... 4
1.5. Amélioration ..... 4
**2. Co-développement Humain-IA** ..... **5**
2.1. Présentation de la stratégie ..... 5
2.2. Sources et support ..... 5
2.3. Contribution Humain - IA ..... 6
2.4. Evaluation critique ..... 6
2.4.1. Dimension quantitative ..... 6
2.4.2. Dimension qualitative ..... 6
**3. Gestion de projet** ..... **7**
3.1. Organisation en sprints ..... 7
3.2. Versionning ..... 7
3.3. Suivi de conformité ..... 7
3.4. Documentation ..... 7
3.5. Qualité du code ..... 7
**4. Annexe : .env** ..... **8**




**Vidéo de présentation** : https://youtu.be/zFgSSeXEGR4

# 1. Description du système

## 1.1. Contexte et objectif

L'application web MJQbe WEB est un hub qui centralise différentes applications en ligne en une interface unique et personnalisable.
L'application propose deux modes d'utilisation : MJ TV, orienté consommation de contenu (streaming, médias), et MJ Desktop, orienté productivité (outils, applications de travail). Les utilisateurs de l'app peuvent posséder le rôle d'utilisateur standard, qui dispose d'un accès en lecture au hub avec personnalisation de son profil et de ses favoris ; ou le rôle d'admin, qui dispose d'un CRUD complet sur les applications, les catégories et les utilisateurs, ainsi que l'accès aux logs d'activité.

## 1.2. Fonctionnalités

| | |
|---|---|
| Authentification | • Inscription / Connexion avec nom d'utilisateur et mot de passe (JWT + Bcrypt)<br>• OAuth 2.0 : connexion via Google ou GitHub<br>• Gestion des rôles : user et admin |
| Hub d'application | • Affichage des applications sous forme de grille ou de liste<br>• Organisation par catégories<br>• Ouverture en iframe intégrée ou nouvel onglet<br>• Filtrage par mode (TV / Desktop)<br>• Recherche<br>• Favoris |
| Interface | • Sidebar fixe avec titre dynamique (MJ TV / MJ Desktop)<br>• Switch de mode TV ↔ Desktop<br>• Responsive (desktop, tablette, mobile)<br>• Pages d'erreur (404, erreur iframe) |
| Personnalisation | • Page Settings : choix du thème, taille des icônes, layout<br>• 14 thèmes |
| Administration | • Logs d'activité : enregistrement des connexions et du lancement des applications (iframe seulement)<br>• CRUD sur les utilisateurs<br>• CRUD sur les applications<br>• CRUD sur les catégories |

| | |
|---|---|
| Profil | • Upload de photo de profil<br>• Changement de pseudo<br>• Ajout / modification d'email<br>• Changement de mot de passe<br>• Vérification d'identité par OAuth ou Mot de passe avant tout changement sensible |

## 1.3. Expérience utilisateur

L'utilisateur commence sur la page de connexion où il peut s'authentifier avec un nom d'utilisateur et un mot de passe, ou plus rapidement via son compte Google ou GitHub.
Une fois connecté, il découvre la page d'accueil du hub. Elle contient une grille d'applications organisées par catégories, dans le mode MJ TV pour accéder à ses plateformes de streaming comme Netflix, YouTube ou Twitch… En cliquant sur une application, elle s'ouvre directement dans la page via une iframe, ou dans un nouvel onglet. Il peut aussi mettre n'importe quelle app en favori pour la retrouver plus facilement. Sur la gauche, une sidebar lui permet de naviguer entre les différentes sections : Accueil, All Apps, Recherche et MJ Desktop, le second mode qui contient des outils de productivité. En bas de la sidebar, il peut aussi accéder aux paramètres. Cela lui permet de personnaliser son expérience : il peut choisir parmi 14 thèmes, ajuster la taille des icônes et le layout. Il peut accéder à sa page de profil pour modifier son avatar, son pseudo, son email ou son mot de passe, avec une vérification d'identité obligatoire avant tout changement sensible. Si l'utilisateur est administrateur, il accède en plus à un panneau où il peut gérer les applications et les catégories du hub, administrer les comptes utilisateurs avec recherche et tri, et consulter les logs d'activité pour suivre toutes les connexions et les lancements d'applications en iframe.

## 1.4. Stratégies FullStack

J'ai appliqué une Clean Architecture en backend (repositories → services → controllers → routes) pour isoler la logique métier. L'authentification repose sur JWT (SPA stateless, 7j) et est complétée par OAuth 2.0 via Passport pour Google et GitHub, avec comptes liés en table dédiée. Côté infrastructure, Docker multi-réseau isole la base de données du public, et le build multi-stage Dockerfile.

## 1.5. Amélioration

Toutes les fonctionnalités du cahier des charges sont implémentées. Parmi les améliorations apportées au-delà du CdC : un affichage des logs plus lisibles (format texte structuré plutôt que JSON brut), la possibilité pour l'utilisateur de sélectionner les catégories et applications qu'il souhaite afficher dans son hub et la documentation avec un swagger. J'aimerai utiliser GitHub Spec Kit et comparer le résultat à l'actuel.

# 2. Co-développement Humain-IA

## 2.1. Présentation de la stratégie

La stratégie que j'ai adoptée repose sur une répartition des rôles : je garde les décisions importantes (architecture, choix techniques) et j'utilise l'IA pour accélérer le développement et garder une trace du projet. Elle agit comme un collègue expérimenté, mais ne me remplace pas : je développe en parallèle.
Je travaille en spec-driven development. À chaque session, l'IA a accès au cahier des charges, aux spécifications, au planning des sprints et à un historique des problèmes. J'ai rédigé ces specs puis je les ai améliorées avec Claude, ChatGPT et Gemini. Elles définissent aussi des règles strictes (par exemple, pas de changement de stack ni d'ajout de dépendances sans validation).
Le code généré n'est jamais intégré tel quel : je le relis, je m'assure de le comprendre et je le valide en fin de sprint. Si besoin, je corrige moi-même ou je demande à l'IA de le faire, dans une logique proche du vibe-coding.
La stratégie repose ainsi sur quatre documents maintenus dans le dépôt :

- specs.md : référence technique principale
- sprints.md : suivi de l'avancement
- problems.md : historique des erreurs et solutions rédigé par l'IA
- themes.md : valeurs des thèmes CSS

Ces documents ont été générés à partir du cahier des charges et du contrat et restructurés avec Claude Plan après la présentation de Mercredi.

## 2.2. Sources et support

| Source | Raison |
|---|---|
| Arthur Bratigny | En premier lieu, je me suis inspirée de sa méthode pour rédiger les spec |
| Thomas Hyaumet, Mathys Geslin | Ils ont validé mes css et proposé des idées pour les améliorer |
| https://youtu.be/z0Hxx4gutBE | Introduction à Claude, Skills, Connecteur et infos sur les agents DIY |
| https://youtu.be/CmrmmszC_F4 | Utilisation de Plan Mode de Claude Code Plan Mode |
| https://youtu.be/rk66k_tuZec | Utilisation de Ralph Wiggum avec Claude. N'a pas pu être appliqué car cela consomme trop de Token. |
| https://youtu.be/-9obEHJkQc8 | Introduction à Github Spec Kit. N'a pas pu être appliqué car cela demande une révision totale du code. |

## 2.3. Contribution Humain - IA

<table>
<tr><td colspan="2">Humain<br><br>20 %</td><td>• Définition initiale du cahier des charges, des contraintes techniques et des règles pour l'agent. Maintien à jour. Rédaction Rapport et slides.<br>• Toutes les décisions d'architecture<br>• Validation et intégration de chaque bloc de code généré<br>• Détection des problèmes de comportement de l'agent (dérives, oublis de contraintes) et correction manuelle ou par instructions<br>• Tests manuels de l'interface et de l'expérience utilisateur<br>• Vérification de la conformité au CdC à chaque fin de sprint<br>• Développement en parallèle, corrigé et repris par Claude a posteriori (gestion JWT, routes, IHM Sidebar, All apps, NotFound,favorites, …)</td></tr>
<tr><td colspan="2">Claude<br><br>75%</td><td>• Génération du code d'implémentation (composants React, routes Express, services, SQL, CSS)<br>• Diagnostic de bugs, via test contre les specs ou à partir de symptômes décrits (tous les cas du problems.md)<br>• Génération de la suite de tests Jest/Vitest à partir des specs<br>• Aide à la rédaction des documents de support : README, ce rapport (reformulation), le journal de problèmes, et un thème pour les slides.</td></tr>
<tr><td rowspan="3">5 %</td><td>Chat GPT</td><td>• Aide à la rédaction du CdC et de ce rapport (raccourcir)<br>• Réponse aux question quand Claude est occupé</td></tr>
<tr><td>Gemini</td><td>• Aide à la rédaction du CdC<br>• Génération du favicon.ico</td></tr>
<tr><td>Figma</td><td>• Création d’une IHM frontend de référence</td></tr>
</table>

## 2.4. Evaluation critique

### 2.4.1. Dimension quantitative

Sur les 9 sprints, l'IA a généré environ 80 % du code brut en termes de lignes. Le temps de développement estimé sans IA pour un projet de cette complexité serait de 4 à 5 semaines à temps plein; la collaboration IA a permis de le réaliser en environ 1 semaine à mi-temps, soit un facteur d'accélération de l'ordre de 8 à 10.

### 2.4.2. Dimension qualitative

**Points forts :** cohérence du code (architecture, conventions) est bonne grâce aux fichiers de contexte, et aucun problème documenté n’a été reproduit .Permet d’éviter les tâches répétitives (ex : CSS)

**Limites :** le code généré peut être verbeux et l’agent oublie parfois des contraintes si la session est longue. La qualité de la collaboration dépend directement des fichiers de contexte, dont le temps de préparation est largement compensé par la cohérence et l’efficacité du projet.

# 3. Gestion de projet

## 3.1. Organisation en sprints

Le projet a été découpé en 9 sprints de durée variable, regroupés en trois releases :

- **Release de base (sprints 1–6) :** fondations Docker, authentification, navigation, layout, gestion des apps, personnalisation UX
- **Release avancée 1 (sprint 7) :** favoris utilisateur, OAuth 2.0 Google/GitHub, système de logs
- **Release avancée 2 (sprint 8) :** 14 thèmes CSS, gestion du profil utilisateur, upload d'avatar, optimisations de performance
- **Release avancée 3 (sprint 9)** : interface admin enrichie (tri, recherche), suite de tests Jest et Vitest

## 3.2. Versionning

J'ai utilisé Git et Github pour enregistrer les différentes versions de mon projet. Pour cela, j'ai demandé à Claude de créer et de pusher à chaque fin de sprint sur une branche de feature par sprint (feature/sprint1-docker, feature/sprint2-auth, etc.). La branche main représente la production stable ; les merges sur dev sont effectuées uniquement à la fin d'un sprint validé. Les issues GitHub tracent chaque tâche des sprints.

## 3.3. Suivi de conformité

La conformité au cahier des charges est vérifiée à la fin de chaque sprint, avec cochage des items dans agents/sprints.md. Les écarts éventuels ont été documentés dans agents/problems.md avec la cause et la solution appliquée.

## 3.4. Documentation

La documentation technique est entièrement incluse dans le dépôt, dans le dossier agents/ :

- specs.md : référence technique complète (source de vérité)
- sprints.md : plan et état d'avancement des sprints
- problems.md : journal des problèmes résolus
- themes.md : documentation des 14 thèmes CSS

specs.md a été crée à partir du cahier des charges et du contrat.
A la racine, le README.md contient un guide d'installation, des variables d'environnement, des commandes utiles, et l'architecture du projet

## 3.5. Qualité du code

J'ai configuré ESLint et il doit passer sans erreur avant chaque commit. Les conventions de nommagesont appliquées de manière uniforme dans le code. Aucune dépendance non listée dans les specs n'a été introduite sans validation. ErrorLens m'a aidé à identifier et corriger rapidement les erreurs dans le code.

# 4. Annexe : .env

COMPLETER LE .ENV COMME SUIT (EXEMPLE) :

```
# PostgreSQL
POSTGRES_USER=user
POSTGRES_PASSWORD=password
POSTGRES_DB=mjqbe

# BDD
APP_ADMIN_USERNAME=admin
APP_ADMIN_PASSWORD=admin123

APP_USER_USERNAME=user
APP_USER_PASSWORD=user123

# Backend
PORT=5000
JWT_SECRET=39d467a268d6e4946307531ebdb822b9074bca81519871b64e065b9
89fec2f9f403ea4748562295dc9029ae6c19b4dab0c404924b86a86172e303f093
52191d7
BCRYPT_ROUNDS=10
CORS_ORIGIN=http://localhost:5173

# Frontend
VITE_API_URL=http://localhost:5000/api

# OAuth Google
GOOGLE_CLIENT_ID=635428465414-vn0i9fbdor03cmu3b287smksm15kedh3.app
s.googleusercontent.com
GOOGLE_CLIENT_SECRET=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
VITE_OAUTH_GOOGLE_ENABLED=true

# OAuth GitHub
GITHUB_CLIENT_ID=Ov23limfCV3kIYpUNvk7
GITHUB_CLIENT_SECRET= 0dc8c3ccf86a5b178ee76d3fd0e3a02326c5edbd
VITE_OAUTH_GITHUB_ENABLED=true

# URL publique du backend (pour les callbacks OAuth)
BACKEND_URL=http://localhost:5000
# URL publique du frontend (pour les redirections OAuth)
FRONTEND_URL=http://localhost:5173
```

→ Cela crée des users par défauts et ajoute OAuth via Google et Github